

Description du logiciel TextPanorama.

Mise en œuvre pratique

1) Description

2) Mise en œuvre pratique

3) Code Python du module principal commenté

1. Description

Le logiciel présenté ici doit permettre de visualiser un ensemble de textes (poèmes, chansons, entretiens). Cet ensemble (pour nous : le corpus) est par ailleurs partitionné en catégories de textes. Pour les chansons d'un auteur, ces catégories seront parfois les "albums" qui regroupent en général les chansons d'une même période (Georges Brassens, Charles Aznavour, Jacques Brel, Leonard Cohen...). Pour l'exemple des 154 sonnets de William Shakespeare, qui sera notre fil conducteur, ce sera une classification de ces sonnets en huit thèmes principaux (topics), classification admise par plusieurs spécialistes de cette œuvre. On consultera, par exemple (www.dtmvic.com/doc/Lebart_JADT_2018.pdf).

Ce texte est aussi repris dans (www.dtmvic.com/doc/Fr_Brassens_Shakespeare_2022.pdf).

Pour une œuvre poétique comme celle de d'Alfred de Vigny, de Victor Hugo, ce pourrait être les recueils ou les publications, qui jalonnent la vie des auteurs. Si le corpus comprend plusieurs auteurs, la classification a priori en catégories peut simplement concerner les auteurs eux-mêmes.

1.1 Que fait le traitement statistique proposé ? Que démontre-t-il ?

(Chaque analyse crée un dossier de résultats (tableaux, figures), daté du jour, de l'heure et de la minute de l'analyse. Dans le cas le plus général, ce dossier "Results" comporte environ 20 graphiques et 12 fichiers texte).

Pour l'essentiel, il fournit des visualisations, par analyse des correspondances (AC), de la table lexicale "mots x textes" (en fonction d'une fréquence minimale des mots). La visualisation simultanée est reprise en ne sélectionnant que les nvt (parametre, nvt = 3 dans l'exemple)

mots les plus caractéristiques de chaque texte. Les catégories (topics, albums, recueils) sont ensuite projetées en éléments supplémentaires dans les plans principaux.

Pour ces points supplémentaires, des zones de confiance Bootstrap (elliptiques, ou enveloppes convexes de répliques) sont tracées, avec ou sans les répliques sur les graphiques à titre de vérification. Il s'agit donc ici d'un Bootstrap partiel, qui teste la valeur statistique de la partition, sans remettre en question l'AC elle-même. (cf. Which Bootstrap for principal axes methods ? In: Selected Contributions in Data Analysis and Classification. P. Brito et al. (eds), Springer, (2007), p 581-588.) (doc/Lebart_Brito_2007.pdf)

Pour compléter l'AC, en essayant de détecter des traits structuraux non visibles sur les premiers plans principaux, un arbre additif des textes est tracé selon l'algorithme qui, expérimentalement, assure la meilleure cohérence avec l'AC. Les catégories supplémentaires sont alors positionnées dans le plan de l'arbre (cf., par exemple, Lebart_JADT2024.pdf).

Puis, des ellipses de confiance et des enveloppes convexes Bootstrap permettent de valider les positions de ces catégories supplémentaires.

1 2. AC et AT : Quelle complémentarité ?

Les méthodes AC et AT ont des vocations différentes car fondées sur des modélisations différentes de la réalité. Rappelons la remarque de Sattath et Tversky (1977) dans un des articles de référence sur les arbres additifs : “ It is interesting to note that tree and spatial models are opposing in the sense that very simple configurations of one model are incompatible with the other model. For example, a square grid in the plane cannot be adequately described by an additive tree” [Sattath S. et Tversky A. (1977). Additive similarity trees. *Psychometrika*, vol. (42), 3: 319-345]. On sait par exemple que l'AC du graphe associé à un damier (grille régulière) reconstitue parfaitement le damier dans le plan principal des axes (1, 2), ce qu'aucune technique de classification ne peut faire. On considérera donc l'AC comme une phase préliminaire indispensable (l'AC permet de reconstituer intégralement les données de départ si l'on prend tous les axes). Dans un article intitulé : « Arbres ou Axes pour les Analyses de Textes ? » (« http://www.dtmvic.com/doc/Lebart_jadt2016.pdf »), nous avons inversement montré que pour décrire des graphes de similarités sémantiques, les AT donnaient de bien meilleurs résultats que l'AC. [cf. aussi : Lebart, Pincemin et Poudat : Analyse des Données Textuelle, Presse de l'Université du Québec, 2019, chap. 8]. Il paraît donc indispensable de mettre en œuvre les deux approches simultanément.

1. 3. Deux options de base

1) La première option de base (ce sera le paramètre : "[wordbag](#)" = 1) transforme les textes en "sacs de mots" (word bags). Elle est souvent appliquée aux textes poétiques, en raison de leurs fréquences lexicales parfois capricieuses. Elle consiste donc à étudier plutôt le vocabulaire (présence ou absence de mots). Voir, par exemple, l'article: "Poems and Songs: What can Textual Data Analysis do?" (www.dtmvic/doc/Lebart_JADT_2022.pdf).

Cette option n'est pas obligatoire, mais nous la citons en premier car, si elle est choisie, elle est évidemment préliminaire à tous les traitements.

2) La seconde option de base proposée (paramètre "[anagreg](#)") a pour effet d'agréger les textes selon la variable catégorielle (topics, albums).

Le cas [anagreg](#) = 1.

Une Analyse des Correspondances du tableau agrégé (catégories x mots) est alors suivie par un arbre additif dont les sommets sont les catégories, avec représentation simultanée des mots et des catégories. Cette option purement descriptive ne comporte pas de validation car les catégories sont actives (et non supplémentaires).

Le cas [anagreg](#) = 0.

C'est le cas le plus général qui donne lieu à beaucoup plus de détails dans les analyses et surtout des phases de validation fondamentales. Dans ce cas, la pertinence des catégories (vis-à-vis du vocabulaire utilisé) est mise en question, et éventuellement validée par les positions des ellipses (ou enveloppes convexes) Bootstrap.

3) Les paramètres suivants sont décrits dans l'interface (Figure 1) avec leurs valeurs par défaut (ces valeurs par défaut peuvent être modifiées puisque le source python de l'interface est disponible).

L'article de base à consulter sur l'ensemble de la démarche est

« www.dtmvic.com/doc/Lebart_JADT_2026.pdf »

(Validation of Simultaneous Additive Trees in Textual Data Visualization (2026)

2. Mise en œuvre pratique de TextPanorama sur l'exemple "Shakespeare/Sonnets".

On suppose que le logiciel Python a été téléchargé (version 3.11 ou + recommandée), ainsi que son éditeur IDLE. On recommande aussi le téléchargement de l'éditeur de texte (libre) NotePad++ qui sera très utile pour traiter les fichiers de textes et de données (notamment pour d'éventuelles conversions ANSI - UTF-8) mais aussi pour éditer les codes Python pour lesquels il propose une coloration syntaxique). La version du logiciel proposée ici fonctionne sur Windows 10 ou +.

2.1 Les modules Python

Toutes les fonctions Python sont dans un dossier de base intitulé "BASIC_TextPanorama". Ce dossier est obtenu en téléchargeant, puis en dézipant le fichier compressé : "BASIC_TextPanorama.zip".

Ce dossier de base comprend le module python d'interface: "Interface_TextPanorama.py", Ce module (interface décrit par la figure 1) déclenchera (bouton : "Execute") l'exécution du module principal "TextPanorama.py".

Le module principal : "TextPanorama.py" fait appel à la bibliothèque de modules contenue dans le dossier : LIB_TextPanorama.

Cette bibliothèque comprend environ 31 modules python, qui seront importés par le module principal. Il est indispensable que ces deux modules ("Interface_TextPanorama.py" et "TextPanorama.py") ainsi que le dossier bibliothèque de modules ("LIB_TextPanorama") soient dans un même dossier, quel que soit d'ailleurs le nom de ce dossier.

Avant toutes choses, dans le dossier « **BASIC_TextPanorama** », ouvrir "**Interface_TextPanorama.py**" avec l'éditeur Python **IDLE**, et choisir : **RUN**, puis **RUN module**. L'interface s'affiche (cf. Figure 1). Il reste à choisir le dossier DATA_Shakespeare, puis les trois fichiers qu'il contient (*voir section 2.2 ci-après*), fichiers dont les noms vont s'afficher une fois que le dossier a été choisi.

Pour cet essai de premier contact, on peut garder toutes les valeurs par défaut des paramètres, et donc passer directement à l'exécution.

Interface for executing TextPanorama.py

1) Selection of folders

Select folder (Folder not yet selected)

2) Select files

Select basic_text (No basic text selected)

Select categories (Categories file not yet selected)

Select identifiers (Categories identifiers file not selected)

3) Select parameters values (Default values)

wordbag
1
If wordbag = 1: Each raw text is replaced with a bag of words; if wordbag = 0: the analyses are performed on raw texts

anagreg
0
If anagreg = 1: Poems/texts are aggregated according to the categories described in category file; if anagreg = 0: Direct analysis of all poems/texts.

freqmin
9
freqmin: Minimum frequency for kept words.

nax
7
nax: Maximum number of axes to keep in the initial correspondence analysis.

nvt
3
nvt: Number of kept characteristic words (criterion: test values)

Detail
0
Detail = 1: For anagreg = 0, detailed outputs (bootstrap replications, various visualizations); Detail = 0: basic outputs.

seuil_vt
2
seuil_vt: Significance threshold of the test-values (for selection of characteristic words).

(Show the parameter values)

4) Execute

Figure 1. Image produite par l'interface « Interface TextPanorama »

2.2 Les trois fichiers de l'exemple Shakespeare :

Les données : dossier [DATA_Shakespeare.zip](#), à télécharger, puis à décompresser.

Le dossier "[DATA_Shakespeare](#)" obtenu contient les 3 fichiers indispensables : le fichier [texte de base](#), le fichiers décrivant les [catégories](#), et le fichier [d'identifiant des catégories](#). Dans le cas d'autres applications, le nom du dossier DATA et les noms des 3 fichiers qu'il contient seront libres, puisqu'ils seront choisis par l'utilisateur dans l'interface.

2.2.1) [Le fichier des textes de base](#), séparateurs de textes "*****" calé en colonne 1.

« ==== » : fin de fichier. Après 4 espaces, l'identificateur du texte : (moins de 12 caractères sans espace)

Esquisse du fichier texte de base "[Sonnet_LowerCase_1_154_2.txt](#)"

**** S01
from fairest creatures we desire increase,
that thereby beauty's rose might never die,
but as the riper should by time decease,
his tender heir might bear his memory:
but thou, contracted to thine own bright eyes,
feed'st thy light'st flame with self-substantial fuel,
.

**** S02
when forty winters shall beseige thy brow,
and dig deep trenches in thy beauty's field,
thy youth's proud livery, so gazed on now,
will be a tatter'd weed, of small worth held:
then being ask'd where all thy beauty lies,
where all the treasure of thy lusty days,
to say, within thine own deep-sunken eyes,
were an all-eating shame and thriftless praise.
.
====

(Ce format est par ailleurs le format texte de base du logiciel DtmVic)

2.2.2) [Le fichier de type ".csv" des catégories](#). (Attention, fichier type Excel avec séparateur virgule ",". Dans les pays où Excel contient des séparateurs "points-virgule", convertir le fichier. Opération très facile avec Notepad++, logiciel libre fortement recommandé ici).

Esquisse du fichier "Categ.csv"

(ici, les 10 premiers sonnets appartiennent à la catégorie "1").

2 colonnes, 155 lignes (154 sonnets + titre)

Titre, T8

S1,1

S2,1

S3,1

S4,1

S5,1

S6,1

S7,1

S8,1

S9,1

S10,1

....

2.2.3) Le petit fichier de type ".csv" des identificateurs de catégories. Même remarque sur les séparateurs ",".

----- fichier "Ident_Categ.csv" -----

Topic8, ident

T1, 1_Procreation

T2, 2_YoungMan

T3, 3_DarkLady

T4, 4_Absence

T5, 5_Storm

T6, 6_Rivalry

T7, 7_Death

T8, 8_Eternal_poetry

Il est souhaitable, dans ce type de fichier, de commencer les identificateurs (secondes colonnes) par un "numéro de catégorie", car sur certaines figures, ces numéros seront repris par les répliques Bootstrap de la catégorie.

2.3 Exécution.

Une fois les trois fichiers de données choisis, dans un premier temps, à titre de premier contact, on peut accepter toutes les valeurs par défaut de l'interface, et donc cliquer sur "**Execute**" en bas du menu de l'interface. Les résultats et les graphiques (que l'on doit systématiquement fermer pour permettre au programme de continuer son exécution) apparaissent dans l'interface IDLE. Notons que tous les graphiques sont sauvegardés dans le dossier « Results_date_heure_minutes) créé à chaque exécution.

3. Code Python du module principal commenté

Ce module est la « colonne vertébrale » du traitement : Sa fonction principale, « **CA_AT_synthesis** » est déclenchée directement à partir de l'interface, et fait appel, directement ou indirectement aux 31 modules de la bibliothèque « **LIB_TextPanorama** ».

Cette fonction principale calcule le tableau lexical de base (mots x textes) procède soit à l'analyse du tableau agrégé (selon le catégories) (quand le paramètre **anagreg = 1**) soit à l'analyse directe des textes, avec successivement (en bref) :

[Analyse du tableau lexical agrégé \(cas : **anagreg = 1**\).](#)

Analyse des correspondances du tableau lexical Catégories x Mots, représentation simultanée Mots et Catégories.

Tracé d'un Arbre Additif dont les sommets sont les catégories. (Tracé selon l'algorithme de Kawada-Kawai). Illustration de l'arbre par les mots (Arbre Additif Simultané).

[Analyse du tableau lexical Textes x Mots \(cas : **anagreg = 0**\).](#)

Analyse des correspondances du tableau lexical Textes x Mots

Projection des catégories (variables supplémentaires ou illustratives) sur les plans principaux.

Ellipses de confiance Bootstrap des positions des catégories.

Tracé d'un Arbre Additif dont les sommets sont les textes. (algorithme de Kawada-Kawai).

Positionnement des catégories (variables supplémentaires ou illustratives) dans le plan du tracé de l'arbre (Arbre Additif Simultané).

Ellipses de confiance Bootstrap des positions des catégories. Toutes ces opérations sont pilotées par un certain nombre de paramètres décrit dans l'interface de la Figure 1.

L'interface **Interface_TextPanorama.py** appelle (bouton : **Execute**) une fonction principale **CA_AT_synthesis()** située dans le module **TextPanorama.py**.

Cette fonction transforme (ou non, en fonction du paramètre « **wordbag** ») les textes initiaux en « sac de mots » (chaque texte est transformé en son « vocabulaire », autrement dit, chaque mot n'apparaît qu'une fois dans un même texte). Cette option est utile pour certains textes poétiques (refrains, répétitions) .

Puis, elle agrège (ou non) les textes par catégories (paramètre « **anagreg** »).

Si les textes sont agrégés (**anagreg = 1**), elle appelle la fonction :

text2tree_Topic(). [brèves descriptions, par AC et AT simultanément, du tableau agrégé Mots x Catégories]

Si les textes ne sont pas agrégés (**anagreg = 0**), elle appelle la fonction :

text2panorama(). [descriptions détaillées, par AC et AT simultanément, du tableau Mots x Textes, avec catégories en supplémentaire et validation Bootstrap des catégories].

Module TextPanorama.py : Python code

```
#
# Description of the file "TextPanorama.py" downloaded as part of the
# folder "BASIC_TextPanorama". Editing directly the file "TextPanorama.py"
# with an editor such as NotePad++ should provide the same syntactic
# coloration.
#-----
# Synthesis: Correspondence Analysis + Simultaneous Additive Tree +
# bootstrap.
#-----
# (here : basic texts with categories (albums or topics) with
# visualizations and bootstrap validation of categories)
# The main module (TextPanorama.py) must be in the same folder as both
# the library folder: 'LIB_TextPanorama', and the interface program
# 'Interface_TextPanorama.py'
#-----
import os
import sys; sys.path.append('LIB_TextPanorama')
    # Making available the library of functions included in the provided
    # folder: LIB_TextPanorama
import numpy as np
#-----
# All the following importations are modules included in the folder :
# 'LIB_TextPanorama'
#-----
import Words_bag as Wb          # Transforming texts into "presence or
                                # absence of words"
import table_lex as tex         # Building a lexical table
import agreg_tablex as agreg   # Aggregating columns of tablex
#
#---- CA ----
import ac_dtm_AT as act         # CA of "tablexfile.csv"
import f_testvall2 as vt       # Computation of test-values
import CA_graf_complet5 as gr5  # CA display + characteristic words
import CA_cat_alb as CA_alb     # CA display + Categories (axes 1, 2)
import Sup_cat_CA as sup_CA     # Coordinates of supplementary
                                # categories (CA)
#
#---- Bootstrap for CA
import CA_cat_alb_boot as catboot # CA with replicates
import CA_cat_alb_boot_ellipses as catbootell # CA with confidence
                                                # ellipses for categ./albums/topics
import CA_cat_alb_hulls as CA_hulls # CA with convex hulls of
                                      # category clouds
import CA_cat_alb_boot_hulls as CA_boot_hulls # CA with convex hulls of
                                                # replicates
#
#---- SAT (Simultaneous Additive Trees)
import Tree_cat_alb_hulls as TreeHulls # Convex hulls of categories
                                      # or albums or topics
import AT_NJ_calc as njt # NJ tree basic computation
import Tree_base as tbase # basic tree (Tree_base.jpg), creating
```

```

# coordinates and identifiers coord_v and
#id_v
import Tree_cat as cat      # Drawing the same tree with categories
                             # numbers as identifiers.

import Forest as fo        # Series of cat-ident trees ( when FOREST = 1,
                             # FOREST being an internal parameter)
import axis1_sign as ax1   # Possible change of axes directions
import w_coord_plot as wcp # Coordinates of words in NJ tree space
import tree_simult_nll as ct # Tree of aggregated texts + words
                             # ("ct" => "Christmas Tree")
import Tree_cat_alb as talb # Tree with categories (alb = album)
import coord_alb as calb   # Coordinates of categories
import ident_alb as new_id # Building extended identifiers
#
#----- Bootstrap for Simultaneous Additive Trees (SAT)
import Tree_cat_alb_boot as talboot # Tree with Bootstrap replicates of
                                     # categories
import coord_alb_boot as calboot    # Coordinates of categories and
                                     # their replicates
import Dupli_csv_boot as Dup        # replicates of categories
import Tree_cat_alb_boot_ellipses as talboot_ellipses
                                     # Tree with Bootstrap confidence ellipses
import Sup_cat_AT as sup_AT         # Coordinates of supplementary
                                     # categories (AT)
import Tree_boot_hulls as TBH       # Tree with Convex Hulls of
                                     # replicates of categories
#
#*****
#
# CA_AT_synthesis (dossier_data, chemin, sentier, piste, wordbag,
# anagreg, freqmin, nax, nvt, Detail, seuil_vt):
#
#-----
# dossier_data : path to the DATA folder
# chemin       : path to the basic text file (within DATA folder)
# sentier      : path to the categories file (within DATA folder)
# piste       : path to the identifiers of categ file (within DATA folder)
#-----
# Parameters
# "wordbag": If wordbag = 1: Each raw text is replaced with a bag of
#             words (frequency = 1 for each word of the text)
#             If wordbag = 0: the analyses are performed on raw texts.
# "anagreg": If anagreg = 1: Poems/texts are a priori aggregated
#             according to the categories described in category file.
#             If anagreg = 0: Direct analysis of all poems/texts.
# "freqmin": Minimum frequency for kept words.If wordbag = 1, the
#             frequency of word W is the number of texts containing W.
# "nax":      Maximum number of axes to keep in the initial CA
#             (correspondence analysis).
# "nvt":      Number of kept characteristic words (criterion: test-
#             values).
# "Detail":   If Detail = 1: For anagreg = 0, detailed outputs

```

```

#           (bootstrap replications, various visualizations).
#           If Detail = 0: basic outputs (default value)
# "seuil_vt": Significance threshold for the test-values (for selection
#           of characteristic words).
#-----
#
def CA_AT_synthesis (dossier_data, chemin, sentier, piste, wordbag,
                    anagreg, freqmin, nax, nvt, Detail, seuil_vt):
    from pathlib import Path
    from datetime import datetime
#-----
# A folder "Results" (with date and hours/min) is created within the
# folder DATA
#-----
    stamp = datetime.now().strftime('%Y-%m-%d_%H-%M')
    input_dir = Path(dossier_data).resolve()
    output_dir = input_dir/("Results_" + stamp)
    Path(output_dir).mkdir(exist_ok = True)
#-----
    output_results = output_dir
#-----
# creating the file "Report.txt" in the folder "Results" created within
# the folder DATA
#-----
    input_dir_new = Path(output_results).resolve()
    ch = input_dir_new/"Report.txt"
#-----
    g = open(ch, "w+")
    print ("\n " + "chemin = " + chemin + " \n" )
    print ("\n " + "sentier = " + sentier + " \n" )
    print ("\n " + "piste = " + piste + " \n" )
    print ("\n " + "wordbag =" + str(wordbag) + ", anagreg = " +
        str(anagreg) + ", freqmin = " + str(freqmin) + "\n" )
    print ("\n " + "nax =" + str(nax) + ", nvt = " + str(nvt) +
        ", Detail = " + str(Detail) + "\n" )
    print ("\n " + "seuil_vt =" + str(seuil_vt) + "\n" )
#----- (report)
    g.write("\n\n " + "chemin = " + chemin + " \n" )
    g.write("\n\n " + "sentier =" + sentier + " \n" )
    g.write("\n\n " + "piste = " + piste + " \n" )
    g.write ("\n\n " + "wordbag =" + str(wordbag) + ", anagreg = "
        + str(anagreg) + ", freqmin = " + str(freqmin) + "\n" )
    g.write ("\n\n " + "nax =" + str(nax) + ", nvt = " + str(nvt)
        + ", Detail = " + str(Detail) + "\n" )
    g.write ("\n\n " + "seuil_vt =" + str(seuil_vt) + "\n" )
#-----
#
#-----**** First decision: Wordbag or not wordbag? (wordbag is
# a parameter)
#-----
    if(wordbag == 1):
        input_dir_new2 = Path(output_results).resolve()
        nomfich = Path(chemin).stem

```

```

    cheminwb = input_dir_new2/(nomfich + "_wb" + ".txt")
# The new wordbag textfile will be in the folder "Results..."
Wb.wb(chemin, cheminwb) # Function wb
#-----

    print ("\n\n " + "A wordbag file is created: Name = " +
          str(cheminwb) + "\n" )
#----- (report)
    g.write("\n\n " + "A wordbag file is created: Name = " +
          str(cheminwb) + "\n" )
#-----
    chemin = cheminwb
#----- end if.
#----- (report)
    ligne = "Step: tablex \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#----- Computation of the Global Lexical Table (Words x Texts)
# (function tablex)
    ch_csv = tex.tablex(output_results, chemin, freqmin)
#-----
#-----**** Second decision aggregation or not of the
# texts according to categories (anagreg is a parameter)
#
    if(anagreg == 1): # will lead to function "text2tree_Topic()"
#
#----- In this option: anagreg = 1, the categories are
# active. They are used to aggregate a priori the texts.
    print ( " Aggregated lexical table \n ")
#
#-----
        text2tree_Topic(ch_csv, output_results, g, freqmin, chemin,
                        sentier, piste, nax, nvt, seuil_vt)
#-----
#
    if(anagreg == 0): # will lead to function "text2panorama ()"
#
#----- In this option: anagreg = 0, the categories are
# supplementary. Their location in the display of the tree will be
# validated through bootstrap replications.
    print ( " Direct analysis of texts (poems, sonnets, etc. \n ")
#-----
        text2panorama (ch_csv, output_results,g, chemin, sentier, piste,
                        freqmin, Detail, nax, nvt, seuil_vt)
#-----
    return
#----- [when anagreg = 1] -----
#-----
# Function "text2tree_Topic()"
#-----
def text2tree_Topic(ch_csv, output_results, g, freqmin, chemin, sentier,
                    piste, nax, nvt, seuil_vt):

```

```

#
# The categories are active. They are used a priori to aggregate the
texts.

#----- (report)
    ligne = "Step: numpy_agreg_new_ok \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
#**** Computation of the aggregated lexical table
#-----
    ch_classic = agreg.agregtablex(ch_csv, output_results, sentier,
    piste)
    lane = ch_classic
#----- (report)
    ligne = "Step: CA_base \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# Classical CA of the aggregated table
#-----
    X, F, fi, fj, psi, psi_u, phi, phi_u, vp = act.AC_base_AT(
        output_results, lane)
    ligne = "Step: CA_plot: rows + columns \n-----\n"
    print (ligne); g.write("\n" + ligne)
#
# Simultaneous graph of aggregated texts and words
#-----
    ax_h, ax_v = 1, 2
    act.graf_simult (output_results, X, psi, phi, ax_h, ax_v)
    ligne = "Step: CA_plot: rows + columns + char. words\n-----
\n" ; print (ligne); g.write("\n" + ligne)
#
# Computation of "test-values" (Selection of characteristic words
# of the categories and texts)
#-----
    num_vt, nvs = vt.Test_Val12(output_results, nvt, seuil_vt, lane)
#----- (report)
    ligne = "Test-values computed \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# Again: Simultaneous CA graph of aggregated texts and words, with
"nvt" most characteristic words linked to their texts
#-----
    gr5.grafACcomplet5 (output_results, X, psi, phi, ax_h, ax_v, nvt,
        num_vt, nvs)
#-----
# Computation of the AT (Additive Tree) - Nearest Joining Neighbors
# method. Vertices : categories
#----- computation of distances
    disMatrix = act.dist(phi, nax)
#----- (report)

```

```

    ligne = "Step: NJ_tree \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
    ident = list(X.columns)
    p = X.shape[1]
#-----
    ch_edge, ch_gml = njt.AT_NJ_calc(output_results, p, disMatrix,
                                    ident)
#----- (report)
    ligne = "NJ_tree computed \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# First drawing of the AT
#-----
    id_v, coord_v = tbase.Treebase(ch_gml, output_results, p)
#----- (report)
    ligne = "NJ_tree first plot \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# (possible change of first axis orientation)
#-----
    aa = ax1.change(coord_v, phi)
    coord_v = aa
#----- (report)
    ligne = "Horiz. orientation of NJ_Tree modified \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# Coordinates of words around the AT
#-----
    coorm = wcp.coor_mots(F, fi, coord_v)
#----- (report)
    ligne = "Words coordinates computed \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# Again: Simultaneous AT graph of aggregated texts and words, with
# "nvt" most characteristic words linked to their texts
#-----
    ct.Tree_simult11 (ch_edge, output_results, F, fi, id_v, coord_v,
                     coorm, nvt, num_vt, nvs)
#----- (report)
    ligne = "Words decorated tree computed and drawn \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
    ligne = "End of the process \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----

```

```

    g.close()
#-----
    return
#
#-----
# End of the function "text2tree_Topic"
#-----
#
#
#[when anagreg = 0]:
#-----
#-----
#           Function "text2panorama()"
#-----
#-----
#
def text2panorama (ch_csv, output_results, g, chemin, sentier, piste,
                  freqmin, Detail, nax, nvt, seuil_vt):
#
# Reminder: In this option (anagreg = 0) the categories are
# supplementary. Their location in the display of the tree are validated
# with bootstrap replications leading to confidence ellipses.
#-----
# Internal control parameter nboot (should be changed within the code)
#-----
    nboot = 30      # The value of 30 (which may seem low) provides good
# results in this specific multidimensional context. The problem is not
# to estimate the value of a numerical variable. Of course, larger
# values could be tested.
#*****
# Internal parameter FOREST (should be changed here, within the code)
# -----
    FOREST = 0      # Producing a "forest" of additive trees
# -----
# FOREST = 0 : Without tree visualizations for each category (or topic
# or album)
# FOREST = 1 : (advisable only if nc, number of categories [or albums,
# topics] is, e.g., < 10)
# If FOREST = 1: Sequence of *nc* ATs displays identified by each
# of the *nc* categories (or *nc* albums). Goal: better visualize both
# the location and the dispersion of each category in the additive tree.
#-----
    print ("\n " + "chemin =" + chemin + "  freqmin = " +
          str(freqmin) + "    nvt = " + str(nvt) + "  nax = " + str(nax) +
          "\n" )
#----- (report)
    g.write("\n\n " + "chemin =" + chemin + "  freqmin = " +
          str(freqmin) + "    nvt = " + str(nvt) + "  nax = " + str(nax) +
          "\n" )
#-----
    ligne = "Step: tablex \n-----\n"
    print (ligne)
    g.write("\n" + ligne)

```

```

#-----
    lane = ch_csv # address of global lexical table Basic_texts x Words
# computed in "CA_AT_synthesis"
#-----
# for some displays : 16 different colors [ first 16] out of 48.
# The maximum number of categories is then < 49, but the following list
# can be extended if necessary
#-----

    colors =
['black','red','green','blue','orange','cyan','steelblue','pink','limegr
een', 'khaki','yellow', 'crimson', 'coral', 'silver', 'olive', 'navy',
'black','red','green','blue','orange','cyan','steelblue','pink','limegre
en', 'khaki','yellow', 'crimson', 'coral', 'silver', 'olive', 'navy',
'black','red','green','blue','orange','cyan','steelblue','pink','limegre
en', 'khaki','yellow', 'crimson', 'coral', 'silver', 'olive', 'navy']
#-----
#tablex has already been saved on files: tablexfile.txt and
tablexfile.csv. Note that tablexfile.csv uses commas as separators (US
convention; for some European countries, commas could be changed into
semicolons, for an XL visualization of the table.
#-----
#----- (report)
    ligne = "Step: CA_base \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
#
# Basic CA : table words x Texts
#-----
# (Reminder: import ac_dtm_AT as act)
    X, F, fi, fj, psi, psi_u, phi, phi_u, vp = act.AC_base_AT(
        output_results, lane)
#--> (Fig CA_1)
#xxxxxxxxxxxxxxxxxxxx
# The coordinates are in the created file: AC_file_tablexfile.csv
#----- (report)
    ligne = "Step: CA_plot: rows + columns \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# Simultaneous Visualization words x Texts with CA plane(1,2)"
#-----
    ax_h, ax_v = 1, 2
    act.graf_simult (output_results, X, psi, phi, ax_h, ax_v)
#-----
#--> (Fig CA_2.1)
#xxxxxxxxxxxxxxxxxxxx
#----- (report)
    ligne = "Step: CA_plot: rows + columns \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
#

```

```

# Simultaneous Visualization words x Texts with CA plane (3,4)"
#-----
    ax_h, ax_v = 3, 4
    act.graf_simult (output_results, X, psi, phi, ax_h, ax_v)
#--> (Fig CA_2.2)
#xxxxxxxxxxxxxxxxxxxxx
#----- (report)
    ligne = "Step: CA_plot: rows + columns + char. words\n-----\n"
    print (ligne)
    g.write("\n" + ligne)
    ligne = "Computation of Test-values \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
#
# fsize : size of font in some displays
    fsize = 1
#***** below: nvt = number of test-values (characteristic words)
    if(nvt > 0):
        num_vt, nvs = vt.Test_Val12(output_results, nvt, seuil_vt, lane)
#----- (report)
        ligne = "Test-values computed (file: Test_values.txt) \n----\n"
        print (ligne)
        g.write("\n" + ligne)
#-----
#
# Visualization in which characteristic words are linked to their Texts.
#-----
#----- (report)
        ligne = "Visualization: texts x Characteristic words \n-----\n"
        print (ligne)
        g.write("\n" + ligne)
#-----
# (Reminder: import graf_AC_complet5 as gr5)
    ax_h, ax_v = 1, 2
    gr5.grafACcomplet5 (output_results, X, psi, phi, ax_h, ax_v,
                        nvt, num_vt, nvs)
#--> (Fig CA_3.a)
#xxxxxxxxxxxxxxxxxxxxx
    ax_h, ax_v = 3, 4
    gr5.grafACcomplet5 (output_results, X, psi, phi, ax_h, ax_v,
                        nvt, num_vt, nvs)
#--> (Fig CA_3.b)
#xxxxxxxxxxxxxxxxxxxxx
#      Reminder:      n,p = X.shape : n words , p Texts
#*****
    ident = list(X.columns)
    p = X.shape[1]
    noident = [""]*len(ident)    # liste vide
#-----
# New identifiers ("albums")for texts (datafile csv: "sentier")
#-----
    id_CA_topic, catv = sup_CA.SupCat_CA(sentier)

```

```

#----- (report)
ligne = "Table id_CA_topic \n-----\n"
print (ligne)
g.write("\n" + ligne)
g.write( str(id_CA_topic))
#-----
ligne = "catv = vector of albums/topics numbers \n-----\n"
print (ligne)
g.write("\n" + ligne)
#-----
# Coordinates of categories (albums/topics) on axes up to 4
naxtot = 4
coord_album_CA, nalbum = calb.coordalbums(catv, phi, naxtot)
#-----
import ident_alb as new_id
#-----
idalb = new_id.identif_alb(piste)
#-----
categ = "_basic ident."
# (Reminder: import CA_cat_alb as CA_alb)
#
ax_h, ax_v = 1, 2
CA_alb.CA_catalb (output_results, categ, catv, ident, phi, fsize,
coord_album_CA, nalbum, idalb, ax_h, ax_v)
#--> (Fig CA_4.a) (axes (1,2))
#xxxxxxxxxxxxxxxxxxxxxxxxxxxx
#
ax_h, ax_v = 3, 4
CA_alb1.CA_catalb1 (output_results, categ, catv, ident, phi, fsize,
coord_album_CA1, nalbum, idalb, ax_h, ax_v)
#--> (Fig CA_4.b) (axes (3,4))
#xxxxxxxxxxxxxxxxxxxxxxxxxxxx
#
#----- Optional change within the code: -----
# The convex hulls of supplementary categories (here: albums/topics)
# emphasize the fact that supplementary categories are DILATED centroids
# of the concerned texts (poems, sonnets, topics...). They could be
# outside the convex hulls of the concerned texts.
#---- (if ok with option, remove the 4 next "#" below to execute)
# ligne = "Convex Hulls of albums (main CA plane) \n-----\n"
# print (ligne)
# g.write("\n" + ligne)
# CA_hulls.CA_catalb_hulls (output_results, catv, id_CA_topic, phi,
fsize, coord_album_CA, nalbum, idalb, categ, colors)
#-----
#
# *** BOOTSTRAP ***** Replicates ****
#*****
#
#-----
#----- (report)
ligne = "Replicates of albums \n-----\n"
print (ligne)

```

```

    g.write("\n" + ligne)
#-----
    pboot, pboot_n, pboot_list = Dup.Dupli_csv2 (output_results, nboot,
sentier)
#*****
#----- (report)
    ligne = "Coordinates of albums + replicates \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
    coord_album_replic, nalbum, ntot = calboot.coordalbums_boot(
        output_results, catv, phi, nboot)
#----- (report)
    ligne = "Identifiers including replicates \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
    idalboot, idalboot_num = talboot.create_idalboot2(nalbum,
        nboot, piste)
#
#-----
# Reminder: "Categories of texts", "albums", "topics", designates the
# same categorical variables in the present code.
#----- (report)
    ligne = "Graph with replicates of albums, and albums" \n-----
    -----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# (Reminder: import CA_cat_alb_boot as catboot)
#-----
# Graph with replicates of albums, and albums
#-----
    xh_album, xv_album = catboot.CAcatalboot (output_results, catv,
ident, phi, fsize, coord_album_replic, nalbum, ntot, idalboot)
#--> (Fig CA_5)
#*****
    catboot.Print_coord_albS_CA(output_results, xh_album, xv_album,
idalboot )
#-----
    nreplic = 1 # presence of replicates in the graphical displays
    nid = 0
    categ = "(nreplic=1_nid=0_basic_ident)"
# (Reminder: import CA_cat_alb_boot_ellipses as catbootell)
#
    list_all = [i for i in range(nalbum)] # Ellipses for all albums
#
    selec_list = list_all
#-----
# Confidence ellipses with replicates and basic text identifiers
#-----

```

```

    xh_album, xv_album = catbootell.CAatalbootellipses (output_results,
selec_list, catv, nid, ident, phi, fsize, coord_album_replic, nalbum,
nboot, idalboot, categ, nreplic, colors)
#
#-->(Fig CA_6_1 - basic identifiers)
#xxxxxxxxxxxxxxxxxxxx
    nreplic = 1 ; nid = 0
#-----
#
    categ = "(nreplic = 1, nid = 0, ident_topic)"
#-----
# Confidence ellipses with replicates and category identifiers
#-----
    xh_album, xv_album = catbootell.CAatalbootellipses (output_results,
selec_list, catv, nid, id_CA_topic, phi, fsize, coord_album_replic,
nalbum, nboot, idalboot, categ, nreplic, colors)
#
#--> (Fig CA_6_2 - identifiers = category numbers- ident_topic)
#xxxxxxxxxxxxxxxxxxxx
#----- Optional change within the code: -----
#   xxxxxxxx Call selection of 2 ellipses xxxxxxxx *** Internal option.
#   Example of categories (topics, albums) 6 and 7.
#   Selection of 2 ellipses from a list: here: [5,6] = topics 6 and 7
#   (remember, origin = 0, instead of 1).
#   This particular selection concerns only the Shakespeare's Sonnets
#   example. To be modified according to possible external information.
#-----
    selec_list = [5,6] # (Albums/topics 6 and 7)
#-----
    categ = "(_selection_of_2_ellipses_replicates)"
    xh_album, xv_album = catbootell.CAatalbootellipses (output_results,
selec_list, catv, nid, id_CA_topic, phi, fsize, coord_album_replic,
nalbum, nboot, idalboot, categ, nreplic, colors)
#--> (Fig CA_6_2_opt1 - identifiers = category numbers- ident_topic)
#xxxxxxxxxxxxxxxxxxxx
    nreplic = 0
    categ = "(_selection_of_2_ellipses_no_replicates)"
    xh_album, xv_album = catbootell.CAatalbootellipses (output_results,
selec_list, catv, nid, id_CA_topic, phi, fsize, coord_album_replic,
nalbum, nboot, idalboot, categ, nreplic, colors)
#--> (Fig CA_6_2_opt2 - identifiers = category numbers- ident_topic)
#xxxxxxxxxxxxxxxxxxxx
#-----
#
    nreplic = 1 ; nid = 1 ; fsize = 1
    categ = "(nreplic = 1, nid =0, basic_ident)"
#
#   ***** Convex hulls of replicates *****
# (Reminder: import CA_cat_alb_boot_hulls as CA_boot_hulls)
#-----
    xh_album, xv_album = CA_boot_hulls.CAatalboot_hulls (output_results,
catv, nid, ident, phi, fsize, coord_album_replic, nalbum, nboot,
idalboot, idalboot_num, categ, nreplic, colors)

```

```

#--> (Fig CA_7)
#xxxxxxxxxxxxxxxxxxx
#-----

#----- (report)
    ligne = "End of CA computations and visualizations \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
*****
#----- (report)
    ligne = "Simultaneous Additive Trees (AT), computations and
visualizations \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#
#-----
# Section Additive Trees      AT  AT  AT  AT  AT  AT  AT -----
#-----
#
# Computation of the distances matrice (distances between texts)
#----- (report)
    ligne = "Computation of distances matrix\n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
#
    disMatrix = act.dist(phi, nax)
    ident = list(X.columns)
    p = X.shape[1]
#----- (report)
    ligne = "Step: Neighbours Joining_Tree \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# Computation of the (Additive Tree) - Nearest Joining Neighbors method.
#-----
    ch_edge, ch_gml = njt.AT_NJ_calc(output_results, p, disMatrix,
                                     ident)
#----- (report)
    ligne = "NJ_tree computed \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
    ligne = "NJ_tree first plot (+ coord_v + id_v) \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
#
# First drawing of the AT.  ( id_v = ident of vertices,
# coord_v = coordinates of vertices)
#-----
    id_v, coord_v = tbase.Treebase(ch_gml, output_results, p)
#-----
#--> (Fig Tree_1_Treebase)

```

```

#xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
#

#----- (report)
    ligne = "New ident for vertices\n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# New identifiers of categories ("albums") for texts (file csv:
"sentier")
    id_AT_topic, catv = sup_AT.SupCat_AT (sentier)
#
#----- (report)
    ligne = "Table / id_AT_topic \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# Copy of identifiers
    g.write( str(id_AT_topic))
#*****
#----- (report)
    ligne = "catv = vector of albums/topics \n-----\n"
    print (ligne); g.write("\n" + ligne)
    g.write(str(catv))
    ligne = "Same tree with new ids \n-----\n"
    print (ligne); g.write("\n" + ligne)
#-----
#*****
# Same AT with ident (topics/albums/categories) instead of basic texts
idents [sonnets/songs/poetries])
    cat.Treecat (output_results, p, id_AT_topic, coord_v)
#--> (Fig Tree_2_Treebase_ident_cat)
#xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
#-----
#----- (report)
    ligne = "Coordinates on AT of albums (texts) \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
    naxtot = 2 # only 2 dimensions (AT is planar)
    coord_album_AT, nalbum = calb.coordalbums(catv, coord_v, naxtot)
#-----
    import ident_alb as new_id
#-----
    idalb = new_id.identif_alb(piste)
# *****
#----- (report)
    ligne = "Basic identifiers for Additive Trees (ident complemented
with blank spaces) \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----

```

```

    ident_AT = ident + [""]*(len(id_AT_topic) - len(ident))
    g.write(str(ident_AT))
#-----
# Same AT with ident "albums" (file csv : piste_boot) + replicates
#----- (report)
    ligne = "Coordinates of albums + replicates \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# (reminder: calboot ...for module: coord_alb_boot.py in the library)
    coord_album_replic_AT, nalbum, ntot =
calboot.coordalbums_boot(output_results, catv, coord_v, nboot)
#
    categ = "(basic_ident)" ; fsize = 1
#-----
    nreplic = 0
#----- (report)
    ligne = "Tree with replicates of albums, and albums (texts) \n-----\n"
    print (ligne)
    g.write("\n" + ligne)
#-----
# (reminder: talboot, for module : Tree_cat_alb_boot.py in the library)
#-----
    xh_album, xv_album = talboot.Treecatalboot (output_results, categ,
catv, ident_AT, coord_v, fsize, coord_album_replic_AT, nalbum, ntot,
idalboot, nreplic)
#--> (Fig Tree_3_1)_Treebase + categories_sup)
#xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    nreplic = 1
#-----
    xh_album, xv_album = talboot.Treecatalboot (output_results, categ,
catv, ident_AT, coord_v, fsize, coord_album_replic_AT, nalbum, ntot,
idalboot, nreplic)
#--> (Fig Tree_3_2)_Treebase + categ_sup + replicates)
#xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
    talboot.Print_coord_albST(output_results, xh_album, xv_album,
idalboot )
#-----
    nreplic = 1
    nid = 0
#-----
    if(Detail > 0):
        categ = "1_Boot_Conf_Ellipses_(nreplic=_1,_ident_topic)"
# Conf. ellipses, identifiers: categories
        list_all = [i for i in range(nalbum)]
        selec_list = list_all #(all categories are selected)
#-----
        xh_album, xv_album = talboot_ellipses.Treecatalbootellipses
(output_results, selec_list, catv, nid, id_AT_topic, coord_v, fsize,
coord_album_replic_AT, nalbum, nboot, idalboot, categ, nreplic, colors)
#--> (Fig Tree_4_1)
#xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

```

```

#-----
    nid = 0
    nreplic = 0
    categ = "2_Boot_Conf_Ellipses_(nreplic=_0,_basic_ident)"
    fsize = 3
# Conf. ellipses, identifiers: basic texts
#-----
    xh_album, xv_album = talboot_ellipses.Treecatalbootellipses
(output_results, selec_list, catv, nid, ident_AT, coord_v, fsize,
coord_album_replic_AT, nalbum, nboot, idalboot, categ, nreplic, colors)
#-----
    nreplic = 0; nid = 0
#--> (Fig Tree_4_2)
#xxxxxxxxxxxxxxxxxxxxxx
#-----
    categ = "3_Boot_Conf_Ellipses_(nreplic=_0,_nid=_0,_ident_topic)"
# Conf. ellipses, identifiers: categories, no replicates
#-----
    xh_album, xv_album = talboot_ellipses.Treecatalbootellipses
(output_results, selec_list, catv, nid, id_AT_topic, coord_v, fsize,
coord_album_replic_AT, nalbum, nboot, idalboot, categ, nreplic, colors)
#--> (Fig Tree_4_3)
#xxxxxxxxxxxxxxxxxxxxxx
#-----
# Convex hulls of replicates.
#-----
    nreplic = 1
    categ = "_ (basic_ident)"
#-----
    TBH.Treecatalboot_hulls (output_results, catv, nid, ident_AT,
coord_v, fsize, coord_album_replic_AT, nalbum, nboot, idalboot,
idalboot_num, categ, nreplic, colors)
#--> (Fig Tree_5)
#xxxxxxxxxxxxxxxxxxxxxx
#-----
#----- Optional change within the code: -----
#*****
# call with select: Internal option (intervention within the code).
# Selection of 2 ellipses from a list: As an example: [5,6] =
confidence ellipses for albums (or topics) 6 and 7 (remember, origin =
0, instead of 1). This is just an example of selection of a pair of
albums (to compare CA and AT ellipses, case of Shakespeare sonnets)
#-----
    selec_list = [5,6] # (Albums 6 and 7)
    categ = "4_(ident_basic_Selection_of_2_ellipses)"
#-----
    xh_album, xv_album = talboot_ellipses.Treecatalbootellipses
(output_results, selec_list, catv, nid, ident_AT, coord_v, fsize,
coord_album_replic_AT, nalbum, nboot, idalboot, categ, nreplic, colors)
#--> (Fig Tree_5.a)
#xxxxxxxxxxxxxxxxxxxxxx
#-----
    categ = "5_(ident_basic_Select_2_ellipses_no_replicates)"

```

```

nreplic = 0
xh_album, xv_album = talboot_ellipses.Treeatalbootellipses
(output_results, selec_list, catv, nid, ident_AT, coord_v, fsize,
coord_album_replic_AT, nalbum, nboot, idalboot, categ, nreplic, colors)
#--> (Fig Tree_5.b)
#xxxxxxxxxxxxxxxxxxxxx
#
#***** FOREST: internal parameter
# Reminder: If FOREST = 1: Sequence of *nc* ATs successively identified
by each of the *nc* categories (*nc* albums). Goal: visualizing both the
position and the dispersion of each category within the tree
#*****
#
    if(FOREST < 0.5):
#----- (report)
        ligne = "End of the process \n-----\n"
        print (ligne)
        g.write("\n" + ligne)
        g.close()
#-----
        return
#-----
#----- (report)
        ligne = "Series of tree with new idents \n-----\n"
        print (ligne)
        g.write("\n" + ligne)
#-----
        fo.forest(output_results, catv, id_AT_topic, p, coord_v)
#----- (report)
        ligne = "End of the process \n-----\n"
        print (ligne)
        g.write("\n" + ligne)
#-----
        g.close()
#-----
        return
#-----
#----- END OF: TexPanorama.py
#-----

```